

PARIS CITE UNIVERSITY

RESEARCH PROJECT

Self-Supervised Test-Time Training for Image Classification

MAY 2026

Authors:

Rayane KHATIM

rayane.khatim@etu.u-paris.fr

22500285

Mehdi AGHAEI

mehdi.aghaei@etu.u-paris.fr

22500371

Supervisor:

Prof. Ayoub Karin

Contents

1	Introduction and Context	2
2	Related Work	2
3	Developed Method	2
4	Implementation	3
5	Experimental Study and Results	3
6	Discussion	5
7	Final Remarks and Future Work	5
8	References	6

1 Introduction and Context

Image classification works well when the test images stay close to the images used during training. In practice, this is not always the case. An image can be noisy, blurry, overexposed, compressed, or simply different from the learned examples. In this case, the model sees a slightly different distribution, and its accuracy can drop a lot.

This project studies this problem with CIFAR-10 and CIFAR-10-C. CIFAR-10 is the clean source domain, while CIFAR-10-C adds standard corruptions such as Gaussian noise, fog, or JPEG compression. The goal is not to create a new deep network, but to test an adaptation strategy at test time.

The main idea is Test-Time Training (TTT). The model receives an unlabeled test batch, makes a small adaptation with a self-supervised loss, and then predicts the labels. In our implementation, the adaptation uses an idea close to ActMAD: aligning the activation statistics of the target batch with the statistics observed on the source data.

2 Related Work

Classic supervised learning trains a model with image-label pairs. For CIFAR-10, this means learning to recognize ten classes from annotated images. This approach is simple and effective, but it assumes that the test images follow a distribution close to the training distribution.

Self-supervised learning tries to learn representations without directly using labels. SimCLR is a contrastive method: two augmented views of the same image should produce close representations, while representations from different images should stay separated [1]. This step gives a backbone that extracts useful features before training a linear classifier.

Test-Time Training uses a self-supervised task during inference to adapt the model to the target distribution [2]. This idea is useful, but it does not always improve the results. Some work shows that success depends on the self-supervised task, the type of distribution shift, and the stability of the adaptation [3].

CIFAR-10-C is a benchmark used to measure robustness to common corruptions [4]. In this project, it is used to compare a baseline prediction without adaptation with a prediction after TTT. The ActMAD part uses internal network statistics, especially in normalization layers, to move the corrupted batch closer to the activations of the source distribution [5].

3 Developed Method

The method followed in the project is organized into four steps.

First, the ResNet18 backbone is pretrained with SimCLR on CIFAR-10. For each image, the loader produces two augmented views. The network passes these two views through the backbone and a projection head. The NT-Xent loss brings the two views of the same image closer and separates the other examples in the batch.

Then, a linear classifier is trained on the features from the frozen backbone. This step gives a simple measure of the quality of the learned representations. The backbone is not modified during

this linear evaluation.

After that, source statistics are computed on the backbone activations. The means and standard deviations of BatchNorm or LayerNorm layers are recorded on the source set. These statistics are saved in `results/self_supervised/source_stats.pt`.

Finally, during testing on CIFAR-10-C, two evaluations are done. The first one is the baseline without adaptation. The second one applies ActMAD TTT: for each target batch, a fresh copy of the model is adapted for a few steps, only on the normalization parameters, and then used to predict the classes.

4 Implementation

The file `run.py` remains the main entry point. It accepts two tasks: `self_supervised` for SimCLR pretraining and `test_time_training` for TTT evaluation. The aliases `ssl` and `ttt` are also kept.

The self-supervised module contains the SimCLR model, the NT-Xent loss, self-supervised training, and linear evaluation. The training loop uses a maximum number of epochs with early stopping. The current configuration is:

```
max_epochs: 32
patience: 12
min_delta: 0.0005
batch_size: 128
```

This keeps early stopping and uses a smaller batch size for easier local runs. The cosine scheduler is called at the end of each epoch, which matches its `T_max` parameter expressed as a number of epochs.

The checkpoints are saved in:

```
results/self_supervised/best_backbone.pt
results/self_supervised/last_backbone.pt
results/self_supervised/simclr_backbone.pt
results/self_supervised/classifier.pt
results/self_supervised/training_log.csv
```

The file `src/test_time_training.py` contains the ActMAD adapter. It keeps the existing logic: copying the model for each batch, freezing most parameters, and adapting the normalization layers with an activation alignment loss.

The dataset module builds the CIFAR-10 and CIFAR-10-C loaders. The metrics module provides top-1 accuracy and baseline/TTT evaluation. The script `eval_corruption.py` evaluates all CIFAR-10-C corruptions and writes a CSV results file.

5 Experimental Study and Results

The experimental protocol was run in three parts. First, SimCLR was pretrained on CIFAR-10. Then, a linear classifier was trained on the frozen backbone to measure the quality of the

learned representation. Finally, the model was evaluated with and without ActMAD-style Test-Time Training on corrupted test images.

The training loss decreased from 5.6214 to 4.7222, which shows that the self-supervised model was learning during training. The strongest improvement happened during the first epochs, while later epochs only gave small gains. The best checkpoint was obtained at epoch 49, not at the final epoch.

The linear evaluation accuracy obtained after training was **59.08%**. This value is not a final state-of-the-art score, but it shows that the backbone learned useful features from the self-supervised pretraining.

For the TTT evaluation, the full CIFAR-10-C files were not installed locally. The code therefore used the fallback implemented in the project: it generated lightweight corrupted versions from the CIFAR-10 test batch. This is useful for testing the method and comparing the baseline with TTT, but it should not be presented as a complete CIFAR-10-C benchmark. Each corruption was evaluated on 10,000 images with severity 5.

Table 1: Baseline and ActMAD TTT results on the evaluated corruptions.

Corruption	Severity	Baseline accuracy	TTT accuracy	Improvement
gaussian_noise	5	16.82%	29.01%	+12.19 pp
shot_noise	5	15.62%	26.39%	+10.77 pp
fog	5	55.76%	54.90%	-0.86 pp
brightness	5	56.51%	52.91%	-3.60 pp
Mean	–	36.18%	40.80%	+4.63 pp

The results show that TTT helped clearly on the two noise corruptions. On **gaussian_noise**, the accuracy increased from 16.82% to 29.01%, which is a gain of 12.19 percentage points. On **shot_noise**, the gain was 10.77 percentage points. These two cases suggest that activation matching can help when the corruption strongly changes the image statistics.

The result is different for **fog** and **brightness**. For fog, the accuracy decreased slightly from 55.76% to 54.90%. For brightness, the decrease was larger, from 56.51% to 52.91%. This means that the adaptation is not always useful. In these cases, the TTT objective may move the normalization statistics in a direction that does not improve classification.

Overall, the mean accuracy increased from 36.18% to 40.80%, with an average gain of 4.63 percentage points on the four evaluated corruptions. This average is positive, but it hides the fact that the improvement depends strongly on the type of corruption.

The real results are saved in:

```
results/test_time_training/results.csv
```

6 Discussion

The first 50-epoch run showed that the loss was still improving near the end, so longer training could still help. The current public config uses a lighter 32-epoch setting with early stopping and batch size 128 so the project can be rerun more easily on a local machine.

The linear evaluation accuracy of 59.08% is acceptable for this small student implementation, especially because the project does not use a large model or a long hyperparameter search. The result also shows the limit of the current setting. The backbone is still weak, because 59.08% linear evaluation accuracy is low for CIFAR-10. SimCLR is sensitive to augmentations, batch size, projection head design, and training length. A better representation could probably be obtained with more careful tuning, but that would make the project heavier and less focused on the TTT part.

The TTT results are mixed, which is important. ActMAD-style adaptation improved the two noise corruptions, especially `gaussian_noise` and `shot_noise`. TTT helps strongly on noise corruptions, but it is not stable on all corruptions. This makes sense because these corruptions strongly disturb the activation statistics, so matching target activations with source statistics can help recover a more useful internal distribution.

However, TTT decreased the accuracy on `fog` and `brightness`. This shows that adaptation at test time is not automatically safe. If the adaptation loss is not well aligned with the classification objective, the model can adapt in the wrong direction. A corrupted batch may contain useful information, but it may also push the normalization parameters away from a better solution.

This motivates safer TTT and tuning the number of adaptation steps. The next experiments compare `steps_per_batch = 3, 5, 10` and severity 3 versus severity 5.

Another limitation is the dataset used in this run. The full CIFAR-10-C archive was not available locally, so the experiment used the fallback synthetic corruptions generated from the CIFAR-10 test set. This is enough to verify that the code runs and that the baseline/TTT comparison works, but a stronger experimental study should use the official CIFAR-10-C files and evaluate all corruptions and several severity levels.

For this reason, the results should be read as a first validation of the implementation, not as a definitive benchmark. The positive average gain is encouraging, but the negative cases are just as important because they show where the method is unstable.

7 Final Remarks and Future Work

The project keeps its original structure: SimCLR, ResNet18, CIFAR-10, CIFAR-10-C style corruptions, and ActMAD Test-Time Training. Pretraining uses an early-stopping ceiling, a best checkpoint, a last checkpoint, and a CSV training log.

The final run gave a linear evaluation accuracy of 59.08%. On the four evaluated corruptions, the baseline reached 36.18% mean accuracy, while TTT reached 40.80%. The average gain was therefore +4.63 percentage points. This improvement came mainly from the two noise corruptions, while fog and brightness became slightly worse after adaptation.

The next step would be to install the official CIFAR-10-C files, run all corruptions, and compare

several severity levels. It would also be useful to test different adaptation steps and batch sizes, because TTT can be unstable when the test batch does not give a good adaptation signal.

8 References

References

- [1] T. Chen, S. Kornblith, M. Norouzi and G. Hinton. *A Simple Framework for Contrastive Learning of Visual Representations*. ICML, 2020.
- [2] Y. Sun, X. Wang, Z. Liu, J. Miller, A. A. Efros and M. Hardt. *Test-Time Training with Self-Supervision for Generalization under Distribution Shifts*. ICML, 2020.
- [3] Y. Liu, P. Kothari, B. Van Delft, B. Bellot-Gurlet, T. Mordan and A. Alahi. *When Does Self-Supervised Test-Time Training Fail or Thrive?* NeurIPS, 2021.
- [4] D. Hendrycks and T. Dietterich. *Benchmarking Neural Network Robustness to Common Corruptions and Perturbations*. ICLR, 2019.
- [5] M. J. Mirza, P. Jané Soneira, W. Lin, M. Kozinski, H. Possegger and H. Bischof. *ActMAD: Activation Matching to Align Distributions for Test-Time Training*. arXiv, 2023.